

Pipes

In **Angular**, **pipes** are a way to transform data in templates. They are a powerful feature that allows you to format and display data in various ways without changing the data itself.

1. Pure and Impure Pipes

Pipes in Angular can be classified into **pure** and **impure** pipes based on their behavior.

- **Pure Pipes:**

- These pipes are only called when Angular detects changes to the **input reference**.
- Angular assumes the pipe output will be the same if the input reference hasn't changed.
- Examples include the built-in pipes like **UpperCasePipe**, **CurrencyPipe**, **DatePipe**, etc.
- **Pure pipes** provide better performance because Angular recalculates the output only when the input changes by reference.

A pipe is pure by default unless explicitly marked otherwise.

- **Impure Pipes:**

- These pipes are called on every change detection cycle, regardless of whether the inputs have changed or not.
- You can create an impure pipe by setting the pure property to false in the pipe decorator.
- They can be useful when working with inputs that change in ways Angular can't detect (e.g., objects with mutable properties, arrays where the contents change but the reference doesn't).

Example:

```
@Pipe({
  name: 'impureExample',
  pure: false
})
export class ImpureExamplePipe implements PipeTransform {
  transform(value: any): any {
    // transformation logic
  }
}
```

2. Using Built-in Pipes

Angular provides several **built-in pipes** to perform common data transformations:

- **DatePipe:** Formats dates according to locale and a specified format.

```
{{ currentDate | date: 'shortDate' }}
```

- **CurrencyPipe:** Formats a number as currency.

```
{{ amount | currency:'USD' }}
```

- **UpperCasePipe/LowerCasePipe:** Converts strings to uppercase or lowercase.

```
{{ name | uppercase }}
```

- **JsonPipe:** Converts a value into a JSON string.

```
{{ dataObject | json }}
```

- **DecimalPipe:** Formats a number as a decimal value.

```
{{ 1.2345 | number: '1.2-2' }} <!-- Output: 1.23 -->
```

3. The PipeTransform Interface

The **PipeTransform interface** is used to define the logic of a custom pipe. When creating a custom pipe, you must implement this interface's transform method.

```
export interface PipeTransform {  
  transform(value: any, ...args: any[]): any;  
}
```

- The value is the input data.
- args are optional additional parameters.
- The method returns the transformed output.

Example: Custom Pipe Implementing PipeTransform

```
import { Pipe, PipeTransform } from '@angular/core';  
  
@Pipe({  
  name: 'multiply',  
  pure: true // By default, pipes are pure  
})  
  
export class MultiplyPipe implements PipeTransform {  
  transform(value: number, multiplier: number): number {  
    return value * multiplier;  
  }  
}
```

Then you can use the custom pipe in your template like this:

```
{{ 5 | multiply:10 }} <!-- Output: 50 -->
```

4. Creating Custom Pipes

Creating custom pipes allows you to tailor data transformations to your specific needs. To create a custom pipe:

1. Create a class that implements the PipeTransform interface.
2. Decorate the class with the @Pipe decorator, which contains a unique name for the pipe.
3. Implement the transform method to define how the input is transformed.

Example: Capitalize Pipe

```
@Pipe({
  name: 'capitalize'
})
export class CapitalizePipe implements PipeTransform {
  transform(value: string): string {
    return value.charAt(0).toUpperCase() +
value.slice(1).toLowerCase();
  }
}
```

Usage in the template:

```
{{ 'hello' | capitalize }} <!-- Output: Hello -->
```

5. Performance Implications of Impure Pipes

While impure pipes can be useful in some scenarios, they can **negatively impact performance** because they are called on **every change detection cycle**, even if no changes have occurred in the input. This can lead to unnecessary recalculations, especially in large applications where change detection happens frequently.

- **Performance Tips:**

- Use **pure pipes** wherever possible to minimize recalculations.
- Only use **impure pipes** when you absolutely need to handle data that changes dynamically or can't be detected by Angular's change detection mechanism.
- If the pipe involves expensive computation or heavy logic, using an impure pipe can degrade the application's performance significantly.

In summary:

- **Pure pipes** are efficient and should be used whenever possible.
- **Impure pipes** should be reserved for rare cases where the data can't be easily handled with Angular's default change detection.

ng new pipeExample

cd pipeExample

ng generate pipe multiply

ng generate pipe filter

ng generate pipe reverse

ng generate pipe message

Explanation of the Example:

1. Built-in Pipes:

- We are using built-in pipes such as **DatePipe** and **UpperCasePipe** to display formatted data.
- today is formatted with **longDate**, and the string '**angular pipes**' is converted to uppercase.

2. MultiplyPipe (Pure):

- This pipe multiplies a number (**numberToMultiply**) by a given factor (multiplier). Since it's a pure pipe, Angular

recalculates the result only when the multiplier or the **numberToMultiply** changes.

3. **FilterPipe (Impure):**

- This pipe filters an array (items) based on a search term (searchTerm). Since it's an impure pipe, Angular recalculates the result every time a change detection cycle runs, even if the input (array reference) hasn't changed.
- When you click the "Add Item" button, new items are added dynamically to the list, and the pipe filters them instantly.

6. **Performance Implications:**

- The **MultiplyPipe** is a **pure pipe**, so it only recalculates when the numberToMultiply or multiplier changes, which improves performance as unnecessary recalculations are avoided.
- The **FilterPipe** is an **impure pipe**, meaning it recalculates on every change detection cycle. This can impact performance negatively, especially with large datasets or frequent change detection cycles.

In this example, the impure pipe is manageable, but in a real-world scenario with more complex operations or larger datasets, it could become a performance bottleneck.

Summary:

- **Pure pipes** like **MultiplyPipe** are recalculated only when inputs change, offering better performance.
 - **Impure pipes** like **FilterPipe** are recalculated on every change detection, which can degrade performance in complex applications.
-

If **var_six** is an array of numbers (**[10, 20, 30, 40, 50]**), the slice filter works similarly to how it operates on strings, but with array elements instead of characters.

Here's how the slicing would work with this array:

Example 1: Slicing with slice:2:-2

<h1>

Sliced Array (2:-2): {{ var_six | slice:2:-2 }}

</h1>

- **Start Index (2):** Starts at the third element (index 2), which is 30.
- **End Index (-2):** Ends 2 elements from the end, which is 40. Since slice excludes the end index, the output will only include the element at index 2.

Output: [30]

Example 2: Slicing with slice:2:-1

<h1>

Sliced Array (2:-1): {{ var_six | slice:2:-1 }}

</h1>

- **Start Index (2):** Starts at the third element, 30.
- **End Index (-1):** Ends 1 element from the end, which is 50. Since slice excludes the last element, it will stop at 40.

Output: [30, 40]

Example 3: Slicing with slice:2:5

<h1>

Sliced Array (2:5): {{ var_six | slice:2:5 }}

</h1>

- **Start Index (2):** Starts at the third element, 30.
- **End Index (5):** Ends before the element at index 5 (which is beyond the length of the array, so it stops at the last element, 50).

Output: [30, 40, 50]

Example 4: Slicing with slice:2:4

<h1>

Sliced Array (2:4): {{ var_six | slice:2:4 }}

</h1>

- **Start Index (2):** Starts at the third element, 30.
- **End Index (4):** Ends before the element at index 4, which is 50. The slice stops before 50.

Output: [30, 40]

How Negative Indices Work with Arrays

- **Negative indices:** When using negative numbers in slicing, the index is counted from the end of the array:
 - **-1: Last element (50 in this case).**
 - **-2: Second-to-last element (40).**
 - **-3: Third-to-last element (30).**

Summary of Outputs for var_six = [10, 20, 30, 40, 50]

- **slice:2:-2** gives [30] (starts at 30, ends at 40).
- **slice:2:-1** gives [30, 40] (starts at 30, ends before 50).
- **slice:2:5** gives [30, 40, 50] (starts at 30, ends at the last element).
- **slice:2:4** gives [30, 40] (starts at 30, ends before 50).

This is how slicing works when applied to arrays of numbers.

In **Angular**, the number filter is used to format numbers according to a specific pattern. The format for this filter is

"minIntegerDigits.minFractionDigits-maxFractionDigits".

Example 1: Number Format (4.1-2): {{ var_seven | number:"4.1-2" }}

<h1>

Number Format (4.1-2): {{ var_seven | number:"4.1-2" }}

</h1>

- **"4.1-2":**
 - **4: Minimum number of integer digits to display.**
 - **1: Minimum number of fraction digits to display.**
 - **2: Maximum number of fraction digits to display.**

Given that var_seven = 100.12345:

- **Integer part:** 100 has 3 digits, but since the minimum integer digits is set to 4, a leading zero will be added, making it **0100**.
- **Fractional part:** Since the number has more than 2 decimal places, it will be rounded to 2 decimal places, i.e., 12 (rounding 0.12345 to 0.12).

Output: 0100.12

Example 2: Number Format (3.2-3): {{ var_seven | number:"3.2-3" }}

<h1>

Number Format (3.2-3): {{ var_seven | number:"3.2-3" }}

</h1>

- **"3.2-3":**
 - **3: Minimum number of integer digits to display.**
 - **2: Minimum number of fraction digits to display.**
 - **3: Maximum number of fraction digits to display.**

For var_seven = 100.12345:

- **Integer part:** 100 has 3 digits, which meets the minimum of 3 integer digits, so no leading zeros are added.
- **Fractional part:** The number is rounded to 3 decimal places, i.e., 123 (rounding 0.12345 to 0.123).

Output: 100.123

Explanation of the Number Filter Syntax

- The **minimum integer digits** ensure the number always has at least the specified number of digits in the integer part, adding leading zeros if necessary.
- The **minimum and maximum fractional digits** specify the number of decimal places. If the number has more decimal places than the maximum, it will be rounded to fit; if it has fewer, zeros will be added to match the minimum.

Summary:

- **"4.1-2"**: Adds a leading zero to make 4 integer digits, rounds to 2 decimal places → 0100.12.
- **"3.2-3"**: Ensures 3 integer digits and rounds to 3 decimal places → 100.123.